

15 - Data Preprocessing and Visualization

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Identify missing values, duplicates, outliers, and inconsistent data types.
- Apply encoding, scaling, and train/test separation without data leakage.
- Use visualizations to inspect distributions and relationships before modeling.

Background

Model quality depends heavily on the quality of the input data. Preprocessing should be reproducible and fit only on training data when it learns statistics. Visualization is not decoration; it is a diagnostic tool for detecting skew, outliers, class imbalance, and suspicious relationships.

Core Concepts

Concept	Meaning in this lab
Missing data	Absent values that may require removal, imputation, or explicit modeling.
Encoding	Conversion of categorical values into a machine-readable numerical representation.
Scaling	Rescaling numeric features for algorithms sensitive to magnitude.
Data leakage	Use of information during training that would not be available at real prediction time.
Exploratory visualization	Plots used to reveal distributions, imbalance, associations, and anomalies.

Guided Procedure

1. Load a tabular CSV and inspect shape, column types, missing counts, duplicates, and summary statistics.
2. Separate the intended target from predictors before preprocessing.
3. Split into training and test sets early.
4. Impute or remove missing values with an explicit rationale.
5. Encode categorical features and scale numeric features where appropriate.
6. Create histograms, box plots, scatter plots, and a correlation view for selected numeric features.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data.csv")
print(df.info())
print(df.isna().sum())

X = df.drop(columns=["target"])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7, stratify=y
)

num_cols = X_train.select_dtypes(include="number").columns
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
```

```
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

Lab Exercises

Task 1. Create a data-quality report containing missing counts, duplicate count, and numeric summary statistics.

Task 2. Compare mean and median imputation on a skewed feature and explain which is more appropriate.

Task 3. Plot class frequencies and discuss whether class imbalance may affect accuracy as a metric.

Task 4. Demonstrate one example of data leakage and then correct it using a train-only fit.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why should preprocessing statistics be learned from training data only?
2. When is one-hot encoding appropriate?
3. What can a box plot reveal that a mean alone cannot?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.